



Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

ЛАБОРАТОРНА РОБОТА № 8
з дисципліни «Технології розроблення програмного забезпечення»
Тема: «Патерни проектування»

Виконала:

Студентка групи ІА-31

Соколова Поліна

Мета: Вивчити структуру шаблонів «Composite», «Flyweight» (Пристосуванець), «Interpreter», «Visitor» та навчитися застосовувати їх в реалізації програмної системи.

15. E-mail клієнт (singleton, builder, decorator, template method, interpreter, client-server)

Поштовий клієнт повинен нагадувати функціонал поштових програм Mozilla Thunderbird, The Bat і т.д. Він повинен сприймати і коректно обробляти pop3/smtp/imap протоколи, мати функції автонастройки основних поштових провайдерів для України (gmail, ukr.net, i.ua), розділяти повідомлення на папки/категорії/важливість, зберігати чернетки незавершених повідомлень, прикріплювати і обробляти прикріплені файли.

Хід роботи

Короткі теоретичні відомості:

Composite (Компонувальник)

Шаблон дозволяє будувати деревоподібні структури, де окремі об'єкти та групи об'єктів обробляються однаково.

Використовується тоді, коли об'єкти утворюють ієрархію “частина—ціле”, і клієнтський код не повинен знати, працює він з елементом чи з групою.

Переваги:

- спрощує роботу з деревами
- дозволяє рекурсивні операції
- клієнтський код не залежить від різниці між Composite і Leaf

Недоліки:

- іноді важко обмежити типи дозволених компонентів
- потребує продуманого інтерфейсу

Flyweight (Легковаговик)

Шаблон дозволяє зменшити кількість об'єктів у системі, розділяючи їх між собою.

Він розділяє стан об'єкта на: внутрішній стан — не змінюється, зберігається всередині Flyweight, зовнішній стан — передається ззовні під час використання.

Використовується там, де є мільйони однакових за структурою об'єктів, наприклад: символи тексту, пікселі, ноти в музиці.

Переваги:

- суттєва економія пам'яті
- прискорення створення однотипних об'єктів

Недоліки:

- ускладнюється код
- частину стану треба передавати кожного разу окремо

Interpreter (Інтерпретатор)

Шаблон використовується для створення власної мови, яка може бути описана у вигляді граматики.

Речення такої мови перетворюються в абстрактне синтаксичне дерево, де кожен вузол - це правило або термін.

Застосовується при створенні: мов правил, фільтрів, регулярних виразів, математичних виразів, пошукових умов

Переваги:

- легко розширювати граматику
- код AST простий і однотипний
- інтуїтивно зрозуміла реалізація

Недоліки:

- не підходить для складних мов

- велика кількість класів при великій граматиці

Visitor (Відвідувач)

Шаблон дозволяє додавати нові операції для об'єктів, не змінюючи ці об'єкти.

Об'єкти приймають "відвідувача", а той вирішує, яку дію виконувати залежно від типу об'єкта.

Застосовується, коли:

треба виконувати багато різних операцій над одними й тими ж класами;

не хочеш змішувати логіку і дані;

легко додати нову операцію, важко — новий тип елемента;

Переваги:

- легко додавати нові операції
- рознесення логіки й структури

Недоліки:

- важко додавати нові типи елементів
- шаблон збільшує кількість класів

Шаблон «Interpreter»

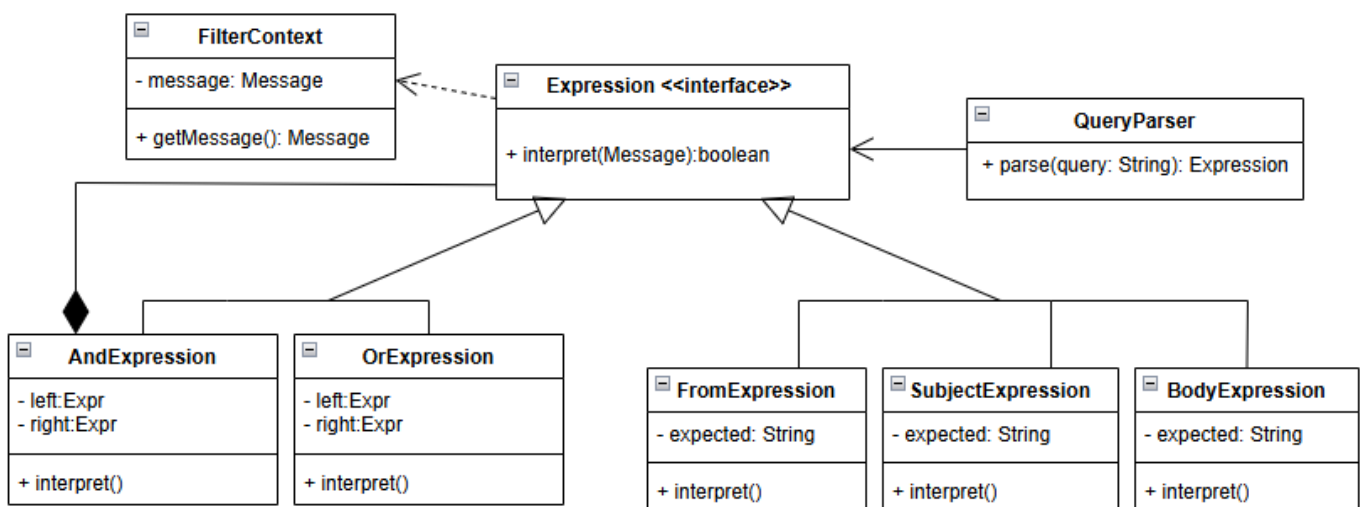


Рис.1 Діаграма класів, яка представляє використання шаблону

На діаграмі зображена реалізація шаблону Interpreter для створення мови фільтрації листів. У центрі схеми знаходиться абстракція AbstractExpression, яка визначає метод interpret(). Від цієї абстракції успадковуються всі класи, які представляють окремі елементи граматики.

Термінальні вирази (TerminalHandlerExpression) відповідають за перевірку конкретного поля листа - теми, відправника, тексту або пріоритету. Вони є базою рекурсії для інтерпретатора. Нетермінальні вирази (AndExpression, OrExpression, NotExpression) описують логіку комбінування фільтрів. Кожен із них містить посилання на інші вирази, що дозволяє створювати складні деревоподібні фільтри.

Клас FilterParser відповідає за побудову абстрактного синтаксичного дерева за введеним текстом пошуку. Після цього MessageFilterInterpreter проходиться по списку листів і визначає, які з них задовольняють вираз. Таким чином, вся логіка пошуку винесена в окремі класи та може легко розширюватися.

```
1 public interface Expression { 22 usages 6 implementations
    boolean interpret(Message msg); 5 usages 6 implementations
}

public class AndExpression implements Expression { 1 usage

    private final Expression left; 2 usages
    private final Expression right; 2 usages

    public AndExpression(Expression left, Expression right) { 1 usage
        this.left = left;
        this.right = right;
    }

    @Override 5 usages
    public boolean interpret(Message msg) {
        return left.interpret(msg) && right.interpret(msg);
    }
}

2 }
```

```
public class OrExpression implements Expression { 1 usage

    private final Expression left; 2 usages
    private final Expression right; 2 usages

    public OrExpression(Expression left, Expression right) {
        this.left = left;
        this.right = right;
    }

    @Override 5 usages
    public boolean interpret(Message msg) {
        return left.interpret(msg) || right.interpret(msg);
    }
}
3 }
```

```
public class BodyExpression implements Expression { 1 usage

    private final String text; 2 usages

    public BodyExpression(String text) { 1 usage
        this.text = text.toLowerCase();
    }

    @Override 5 usages
    public boolean interpret(Message msg) {
        return msg.getBody() != null &&
            msg.getBody().toLowerCase().contains(text);
    }
}
4 }
```

```
public class FromExpression implements Expression { 1 usage

    private final String expected; 2 usages

    public FromExpression(String expected) { 1 usage
        this.expected = expected;
    }

    @Override 5 usages
    public boolean interpret(Message msg) {
        return msg.getSender() != null &&
            msg.getSender().equalsIgnoreCase(expected);
    }
}
5 }
```

```

public class PriorityExpression implements Expression { 1 usage

    private final Priority priority; 2 usages

    public PriorityExpression(String p) { 1 usage
        this.priority = Priority.valueOf(p.toUpperCase());
    }

    @Override 5 usages
    public boolean interpret(Message msg) {
        return msg.getPriority() == priority;
    }
}
6 }

```

```

public class SubjectExpression implements Expression { 1 usage

    private final String text; 2 usages

    public SubjectExpression(String text) { 1 usage
        this.text = text.toLowerCase();
    }

    @Override 5 usages
    public boolean interpret(Message msg) {
        return msg.getSubject() != null &&
            msg.getSubject().toLowerCase().contains(text);
    }
}
7 }

```

```

public class FilterContext { 3 usages

    private final List<Message> messages; 2 usages

    public FilterContext(List<Message> messages) { 1 usage
        this.messages = messages;
    }

    public List<Message> filter(Expression expr) { 1 usage
        return messages.stream()
            .filter(expr::interpret)
            .toList();
    }
}
8 }

```

```

public class QueryParser { 3 usages

    public Expression parse(String query) { 5 usages
        query = query.trim().replace( target: " ", replacement: "");

        // AND
        if (query.contains("&")) {
            String[] parts = query.split( regex: "&", limit: 2);
            return new AndExpression(parse(parts[0]), parse(parts[1]));
        }
        // OR
        if (query.contains("|")) {
            String[] parts = query.split( regex: "\\|", limit: 2);
            return new OrExpression(parse(parts[0]), parse(parts[1]));
        }
        // Terminal rules
        if (query.startsWith("from:"))
            return new FromExpression(query.substring( beginIndex: 5));

        if (query.startsWith("sub:"))
            return new SubjectExpression(query.substring( beginIndex: 4));

        if (query.startsWith("body:"))
            return new BodyExpression(query.substring( beginIndex: 5));

        if (query.startsWith("prio:"))
            return new PriorityExpression(query.substring( beginIndex: 5));

        throw new IllegalArgumentException("Unknown expression: " + query);
    }
}
9 }

```

Рис.2 Фрагменти коду по реалізації шаблону

(1– Expression, 2 – AndExpression, 3 – OrExpression, 4 – BodyExpression,
5 – FromExpression, 6 – PriorityExpression, 7 – SubjectExpression,
8 – FilterContext, 9 – QueryParser)

Використання шаблону дозволило відокремити механізм визначення фільтрів від основного коду інтерфейсу та сервісів. Це спростило реалізацію, зробило її розширюваною та більш гнучкою.

Висновок: у даній лабораторній роботі було реалізовано шаблон «Interpreter», який дозволив створити власну мову фільтрації поштових повідомлень. Завдяки впровадженню термінальних і нетермінальних виразів, система навчилася будувати та інтерпретувати абстрактне синтаксичне дерево, що забезпечило гнучкий і розширюваний пошук листів за складними умовами.

Відповіді на питання:

1. Призначення шаблону «Композит».

Шаблон призначений для подання об'єктів у вигляді деревоподібної структури, де окремі елементи й композиції елементів мають спільний інтерфейс. Завдяки цьому клієнтський код може працювати з одиничними та складними об'єктами однаково.

2. Структура шаблону «Композит».

У структурі беруть участь три основні компоненти: абстрактний компонент, що визначає інтерфейс; лист, який представляє кінцевий елемент без дочірніх вузлів; та композит, який містить інші компоненти та делегує їм виконання операцій рекурсивно.

3. Які класи входять у шаблон «Композит» і як взаємодіють?

Абстрактний компонент визначає загальний інтерфейс. Класи-листи реалізують цей інтерфейс і представляють атомарні об'єкти. Клас-композит також реалізує цей інтерфейс, але містить колекцію дочірніх компонентів і виконує операції шляхом обходу цієї колекції.

4. Призначення шаблону «Flyweight».

Його призначення — економія пам'яті за рахунок повторного використання об'єктів, які мають незмінний внутрішній стан. Це особливо актуально для систем, що оперують великою кількістю дрібних однотипних об'єктів.

5. Структура шаблону «Flyweight».

Вона включає сам об'єкт-легковаговик, який містить внутрішній стан; фабрику, що створює і кешує легковаговики; а також клієнтський код, який передає зовнішній стан під час використання цих об'єктів.

6. Які класи входять у шаблон «Flyweight» і яка між ними взаємодія?

Є інтерфейс Flyweight, конкретний Flyweight, що реалізує незмінний стан, фабрика, яка керує створенням і повторним використанням екземплярів, і клієнт, який надає зовнішні параметри під час роботи.

7. Призначення шаблону «Інтерпретатор».

Шаблон дозволяє формально описати граматику простої мови та створити інтерпретатор, який обчислює речення цієї мови на основі абстрактного синтаксичного дерева.

8. Призначення шаблону «Відвідувач».

Він використовується для додавання нових операцій над наборами об'єктів без зміни класів цих об'єктів. Це досягається завдяки введенню окремих об'єктів-відвідувачів, які реалізують операції для кожного типу елементів.

10. Структура та взаємодія у шаблоні «Відвідувач».

Структура складається з інтерфейсу Visitor, в якому оголошуються методи для кожного типу елементів; конкретних відвідувачів, що реалізують поведінку; інтерфейсу Element із методом асерт; та конкретних елементів, які викликають відповідний метод відвідувача.